Assignment 3: Neural Networks

Task: CIFAR-10 classification

Group member 1: Anshul Kaushal (2611AI10)
Group member 2: Aditya Sharma (2611CS11)
Group member 3: Roland Zohmingthana (2611MC03)

Abstract

The supplied notebook implements a fully connected neural network and manual backpropagation, trains six activation/width configurations on a fixed CIFAR-10 subset, logs run metrics to Weights & Biases (W&B), selects a checkpoint using validation accuracy, and evaluates it on the official test set. The selected configuration is **3072–128–10 with ReLU**: 394,634 trainable parameters, 38.35% best validation accuracy at epoch 10, and **39.51% test accuracy** (test loss 1.7027). The report includes every plot embedded in the supplied notebook plus W&B metric plots reconstructed from the per-epoch records saved in the notebook.

Important completion findings. The notebook's written activation/width interpretation conflicts with its own results table; this report corrects the interpretation. The original notebook also lacks an explicit limitations paragraph, dependency/execution instructions, and the required group-member details. The report supplies a limitations discussion but the group details must be added by the team to this report and the notebook before submission.

1 Dataset and preprocessing

CIFAR-10 has 50,000 official training images and 10,000 official test images, each an RGB image of size 32×32 with ten classes. The notebook uses Keras only to load CIFAR-10, then applies NumPy preprocessing:

- A single `np.random.default_rng(42)` is used while processing labels 0–9 in class order. For each class, 1,000 original training indices are sampled without replacement; the first 800 selected examples are assigned to training and the remaining 200 to validation.
- The same resulting subsets are reused in all six experiments: 8,000 training examples and 2,000 validation examples. The official 10,000-image test set is reserved for final evaluation.
- Pixel values are converted to floating point and divided by 255, retaining all RGB channels and flattening each image to 3,072 features. No further normalization is applied.

2 Network and training method

The model implementation supports one or two hidden layers, user-specified hidden widths, and either ReLU or sigmoid activation. The output layer has ten logits and uses a numerically stable softmax: subtract the row maximum before exponentiation. The parameter initialization is documented as $W \sim \mathcal{N}(0, 1/n_{\text{in}})$ (standard deviation $1/\sqrt{n_{\text{in}}}$), with zero biases and seeded NumPy generators.

For the required example architecture $3072 \to 64 \to 10$, parameter shapes are $W_1 \in \mathbb{R}^{64\times3072}$, $b_1 \in \mathbb{R}^{64}$, $W_2 \in \mathbb{R}^{10\times64}$ and $b_2 \in \mathbb{R}^{10}$. The trainable parameter count is

$$(3072 \cdot 64 + 64) + (64 \cdot 10 + 10) = \mathbf{197,322}.$$

The notebook implements the mean categorical cross-entropy and its gradient manually using a shifted log-softmax expression; each linear layer accumulates gradients for its weights and biases. Its optimizer supports ordinary mini-batch SGD (zero momentum) and momentum SGD. The six experiments use momentum 0.9, learning rate 0.01, batch size 128, 10 epochs, initialization seed 42 and shuffle seed 2026. Training data is shuffled each epoch, and complete training/validation loss and accuracy are computed after each epoch.

3 Experiment results and model selection

All non-width hyperparameters were held fixed across the six runs. The best-validation-accuracy selection rule is implemented using strict improvement within each run, retaining the earliest epoch if an accuracy is tied; across runs, the code sorts by validation accuracy, parameter count and run order. There were no cross-run validation-accuracy ties in the reported results.

Table 1: W&B experiment summary recorded by the notebook. Training time is total elapsed time measured with `time.perf_counter` inside `train_model`, from immediately before the epoch loop through its completion; it includes mini-batch training, full-subset train/validation evaluation, W&B logging and per-epoch reporting, but excludes dataset loading and model construction.

Run	Width	Activation	Parameters	Best val. acc.	Best epoch	Time (s)
relu_width_32	32	ReLU	98,666	35.35%	10	6.1142
sigmoid_width_32	32	Sigmoid	98,666	34.65%	10	3.1620
relu_width_64	64	ReLU	197,322	37.65%	5	4.7117
sigmoid_width_64	64	Sigmoid	197,322	35.15%	10	6.4072
relu_width_128	128	ReLU	394,634	**38.35%**	10	14.3142
sigmoid_width_128	128	Sigmoid	394,634	35.40%	10	13.6539

Selected model: `relu_width_128`, architecture 3072 → 128 → 10, 394,634 parameters, best epoch 10. Its checkpoint is saved when validation accuracy reaches a new best value and is restored before test-set evaluation. The W&B project is available at **apr-assignment-3-neural-networks**.

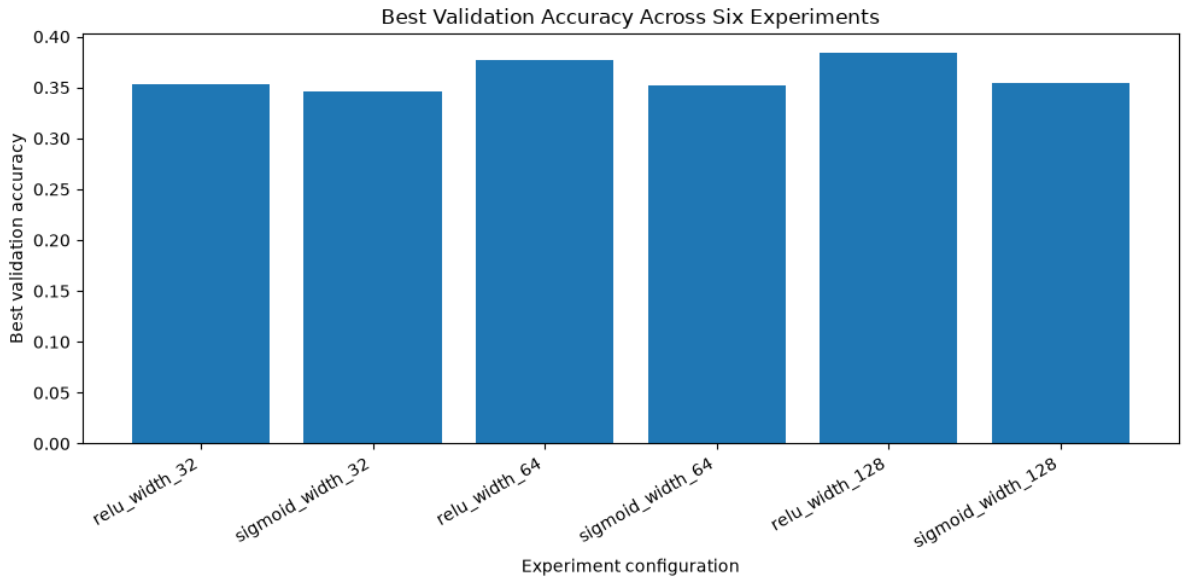


Figure 1: Notebook figure: best validation accuracy across the six configurations. This exact image is taken from the saved notebook output.

4 W&B metric plots

W&B receives the metrics `train/loss`, `train/accuracy`, `validation/loss`, `validation/accuracy` and `epoch_seconds` at every epoch. The supplied notebook stores run links, summaries and per-epoch console records for all six configurations. The following aggregate charts reconstruct those W&B metrics from the saved per-epoch text; loss/accuracy values are shown at the four-decimal precision printed in the notebook, while epoch times are shown at the two-decimal precision printed there. The live Forge dashboard could not be independently rendered in this session, so these are faithful reconstructions of the available run records rather than screenshots exported from the live dashboard.

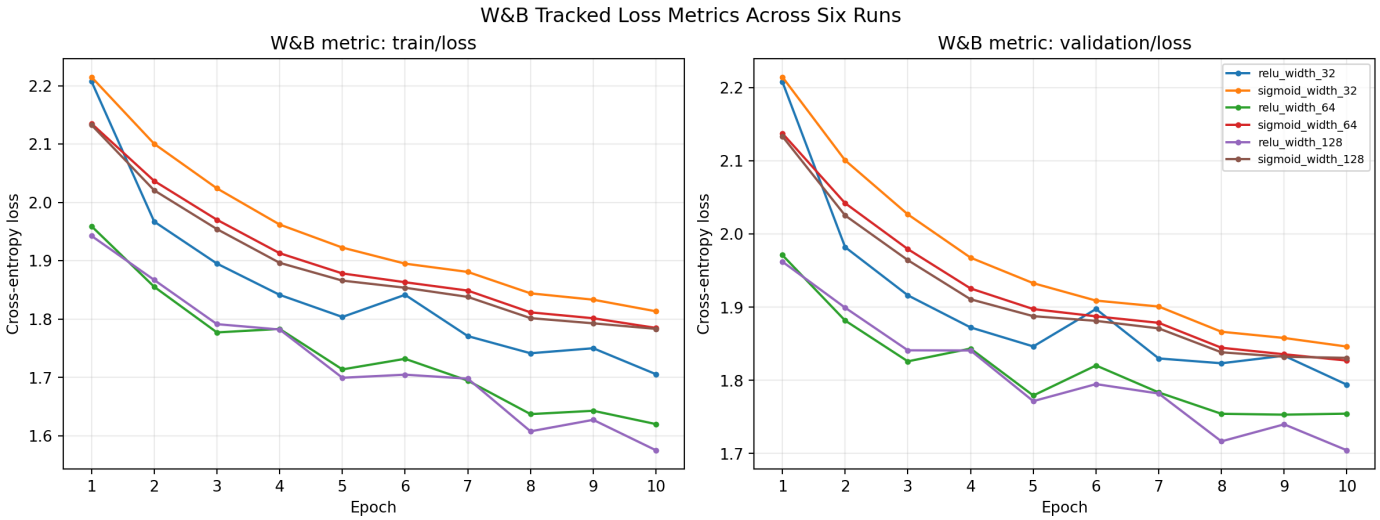


Figure 2: W&B loss metrics across all six runs. Both training and validation loss trend downward overall; the ReLU width-128 run finishes with the lowest reported validation loss among the runs (1.7044).

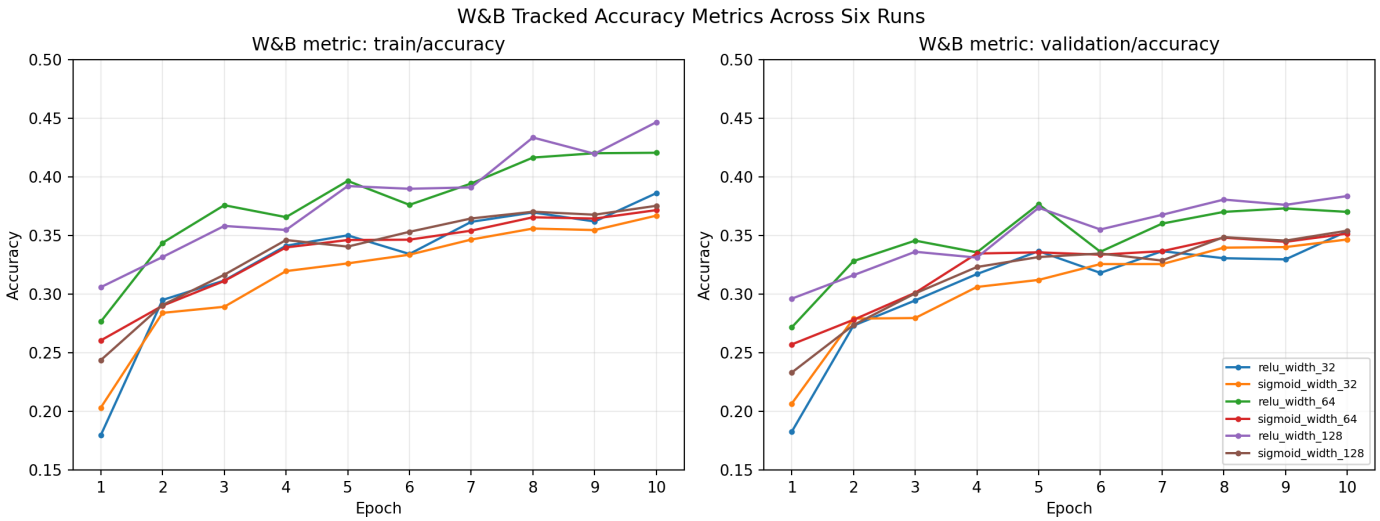


Figure 3: W&B accuracy metrics across all six runs. ReLU configurations outperform their sigmoid counterparts at each tested width; the best validation accuracy is 38.35%.

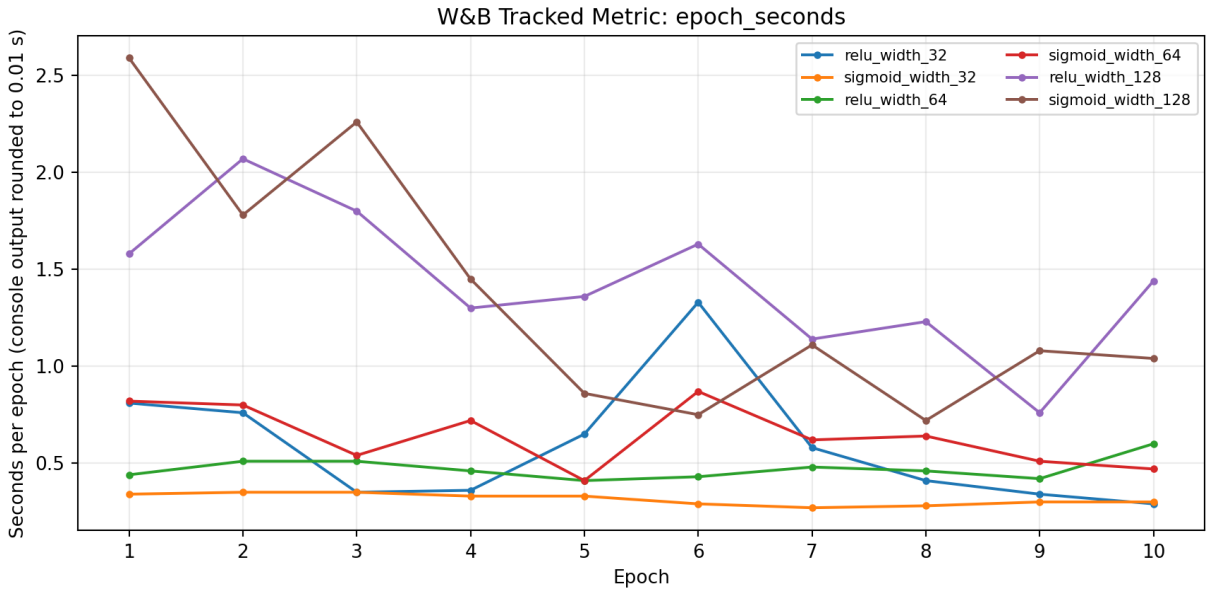
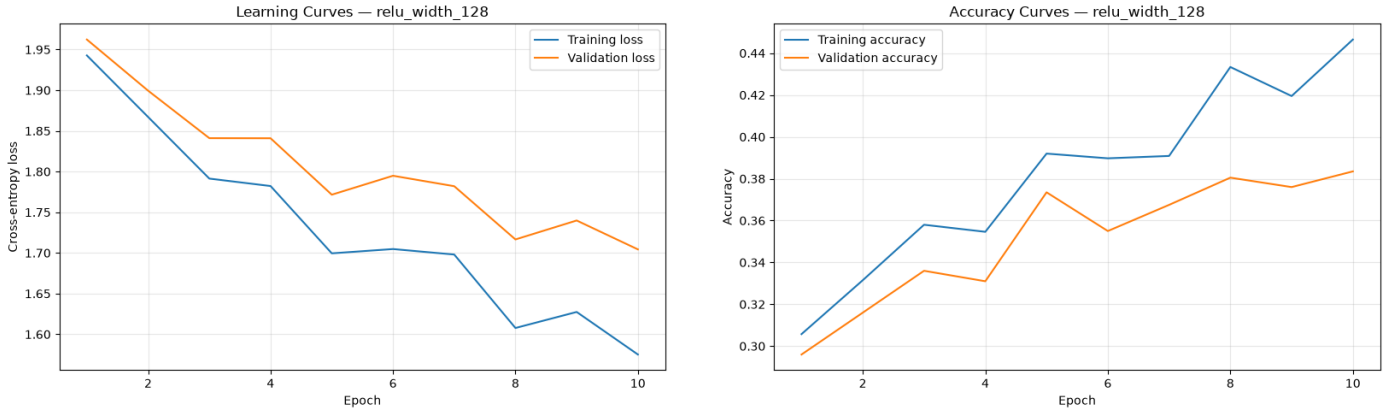


Figure 4: W&B `epoch_seconds` metric across runs. Per-epoch times vary with model width and runtime conditions; total run times in Table 1 are measured separately across the full training function.

5 Selected-model learning curves



(a) Training and validation cross-entropy loss. **(b)** Training and validation accuracy.

Figure 5: The two original notebook plots for the selected ReLU width-128 checkpoint. Training loss declines from about 1.94 to 1.58, while validation loss declines from about 1.96 to 1.70 with some epoch-to-epoch fluctuation. Training accuracy reaches 44.65% and validation accuracy reaches 38.35%.

Interpretation of width and activation

The result table supports the following conclusions:

- **Activation: ReLU wins at every width.** At width 32, ReLU achieves 35.35% vs. sigmoid's 34.65%; at width 64, 37.65% vs. 35.15%; and at width 128, 38.35% vs. 35.40%.
- **Width: larger hidden layers help in these runs.** ReLU improves from 35.35% to 37.65% to 38.35% as width grows from 32 to 64 to 128. Sigmoid also rises slightly from 34.65% to 35.15% to 35.40%; it does not fall at width 128 in the reported results.
- **Generalization: a moderate gap remains.** At the selected model's final/best epoch, training accuracy is 44.65% and validation accuracy is 38.35%, a 6.30 percentage-point gap. Training loss is 1.5752 versus validation loss 1.7044. The validation loss generally declines rather than showing a sustained late-epoch rise, so the curves indicate learning with a generalization gap, but not strong evidence of severe late-stage overfitting.

Correction to the notebook narrative: the notebook text states that sigmoid is better at width 32 and that sigmoid accuracy decreases at width 128. Both claims contradict the run-summary table and chart. The numerical results above are used here as the source of truth.

6 Test-set evaluation

The selected checkpoint was restored before evaluating all 10,000 official test images; the test set was not used for tuning. The recorded results are **test loss = 1.7027** and **test accuracy = 39.51%**. The confusion matrix below is the exact image embedded in the notebook. The diagonal sums to 3,951 correct predictions out of 10,000 images.

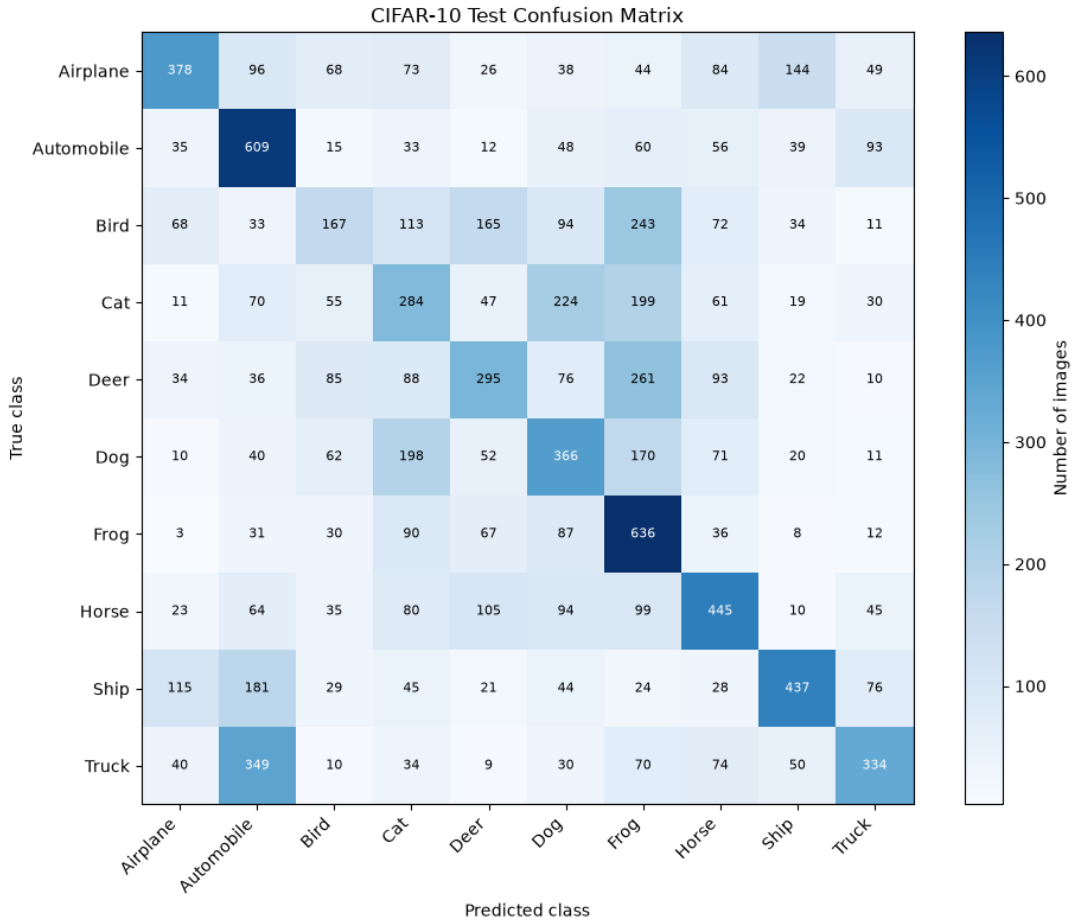


Figure 6: 10-by-10 confusion matrix from the original notebook. Rows are true labels, columns are predicted labels, and class names are shown on both axes. The model is strongest on Frog (636 correct) and Automobile (609 correct); frequent confusions include Truck predicted as Automobile (349 cases), Cat predicted as Dog (224), and Bird predicted as Frog (243).

Limitations of flattened fully connected models

Flattening the $32 \times 32 \times 3$ image turns spatially adjacent pixels into entries of a 3,072-element vector and does not explicitly preserve two-dimensional neighbourhood structure. A dense layer must learn useful spatial relationships from data without convolutional weight sharing or built-in translation equivariance, and its parameter count grows quickly with hidden width. Consequently, this permitted fully connected baseline can confuse visually similar classes and is less suited to local edges, textures and spatial patterns than an architecture designed to exploit image structure. The result is also limited by the small 8,000-example training subset and the fixed 10-epoch experiment budget.